ONLINE BOUTIQUE ON AMAZON EKS

Technical Design Document

How the platform is built and why

Document ID	OBE-TDD-001
Version	2.1
Date	2026-10-10
Status	Approved for the dev environment
Owner	khaipd18 (DevOps / Cloud)
Audience	Cloud and DevOps engineers, reviewers
Classification	Internal
Repository	https://github.com/khaipd18/online-boutique-gitops-pipeline

Document control

Revision history

Version	Date	Change	Author
1.0	2026-10-09	First issue.	khaipd18
1.1	2026-10-10	First run on the new account; managed log group.	khaipd18
1.2	2026-10-10	Full EKS deployment, screenshots, access entries, three nodes, decision D11.	khaipd18
2.0	2026-10-10	Rewritten to be shorter: one page per area, repeated content removed.	khaipd18
2.1	2026-10-10	GitOps write-back through pull requests under the main ruleset.	khaipd18

Related documents

- OBE-RUN-001 Operations Runbook: how to run and fix the platform
- `docs/infrastructure/`: editable HLD/LLD diagrams and their YAML specs

Contents

1 Overview 3
2 AWS infrastructure 4
2.1 Network 4
2.2 EKS, ECR and state 4
2.3 Access for CI (GitHub OIDC) 5
3 Kubernetes platform 6
4 Delivery pipeline 8
5 Security 9
6 Design decisions 10
7 Verification 11
8 Risks and roadmap 12
Appendix. Versions 13

1 Overview

What it is. Google's Online Boutique (10 microservices in 5 languages, gRPC between them) running on Amazon EKS, with everything around it built in this repository: infrastructure as code, CI/CD, GitOps and security controls. The application code in `src/` is upstream code and is not changed.

Goals.

- No long-lived AWS keys anywhere; only reviewed code can change AWS.
- Misconfigurations and vulnerable images are caught before they are deployed.
- The cluster can be rebuilt from Git alone; moving to another AWS account needs no code change.
- A dev environment that is cheap to start and to stop.

Status. Deployed and tested end to end on EKS on 2026-10-10 (section 7), then destroyed to save cost. One environment (`dev`) in `ap-southeast-1`.

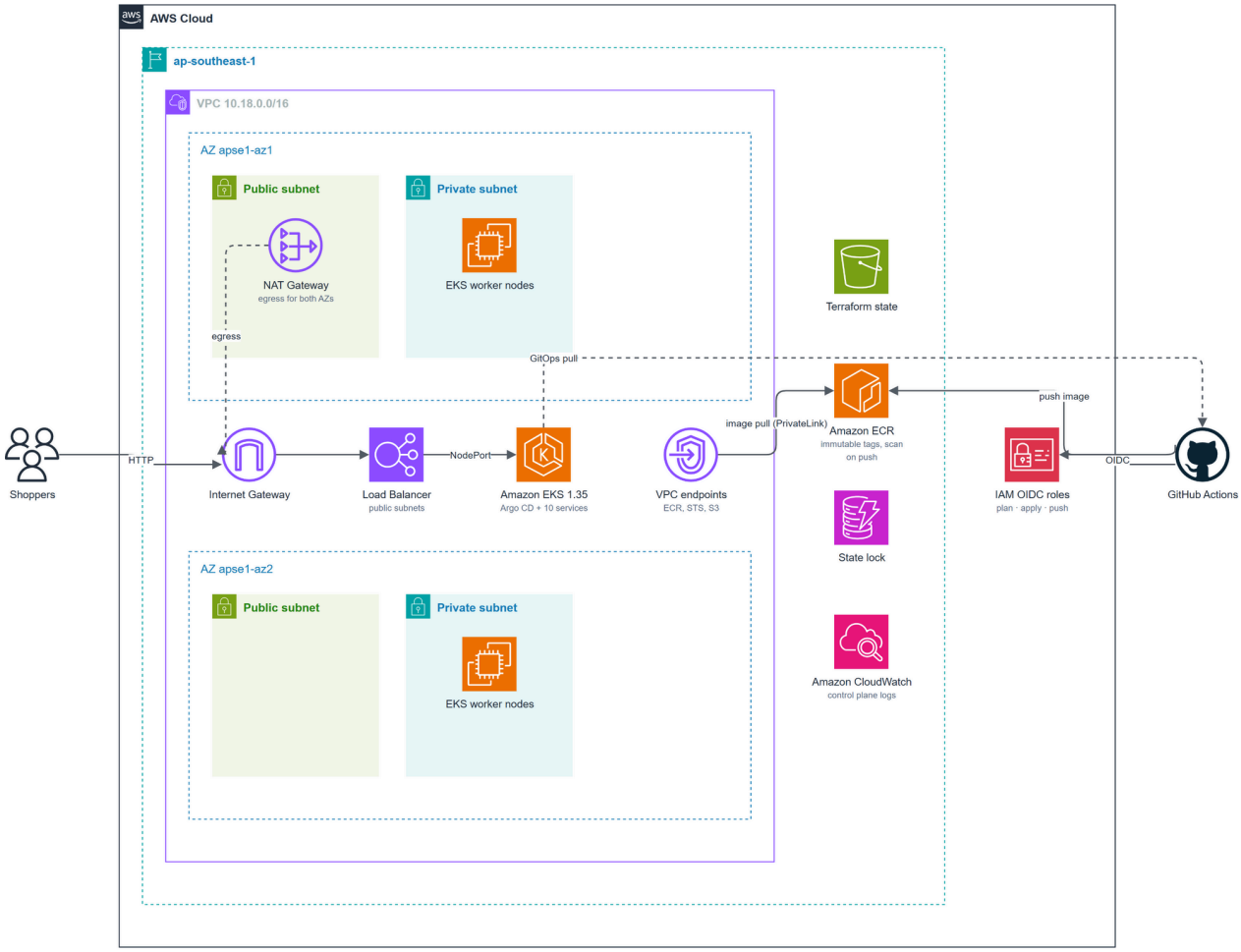


Figure 1: High-level architecture

Users reach the shop through a load balancer in the public subnets; the worker nodes sit in private subnets. Nodes reach ECR, STS and S3 through VPC endpoints and the internet through one NAT Gateway. Argo CD inside the cluster pulls the desired state from GitHub.

2 AWS infrastructure

2.1 Network

Subnet	AZ ID	CIDR	Contains
public (az1)	apse1-az1	10.18.0.0/24	NAT Gateway, load balancer
public (az2)	apse1-az2	10.18.1.0/24	load balancer
private (az1)	apse1-az1	10.18.2.0/24	EKS nodes, interface endpoints
private (az2)	apse1-az2	10.18.3.0/24	EKS nodes, interface endpoints

- VPC `10.18.0.0/16` in two AZs, placed by AZ ID because AZ names differ between accounts.
- One NAT Gateway for both AZs to save cost (decision D3).
- VPC endpoints for `ecr.api`, `ecr.dkr`, `sts` and S3, so image pulls stay inside the VPC [AWS].
- Traffic is filtered by security groups and Kubernetes NetworkPolicy; the network ACLs allow everything.

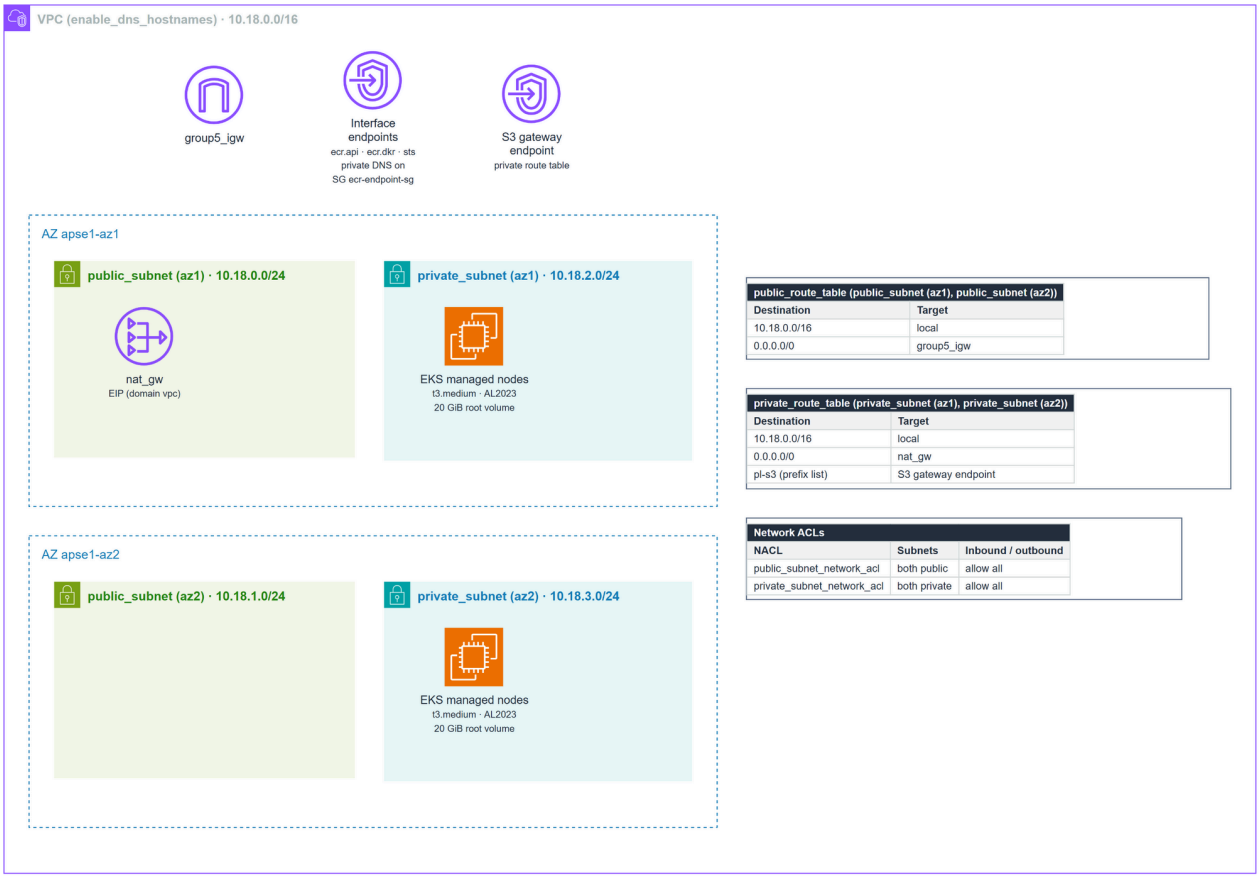


Figure 2: LLD: network detail

2.2 EKS, ECR and state

Part	Design
EKS cluster	`khaipd18-eks-cluster`, Kubernetes 1.35 (standard support until 27 March 2027 [AWS]); public and private API endpoint; all control plane logs to CloudWatch (365 days)
Nodes	Managed node group, 3 × `t3.medium` (min 1, max 4), Amazon Linux 2023, private subnets only
Add-ons	VPC CNI with NetworkPolicy enforcement, CoreDNS, kube-proxy, Metrics Server

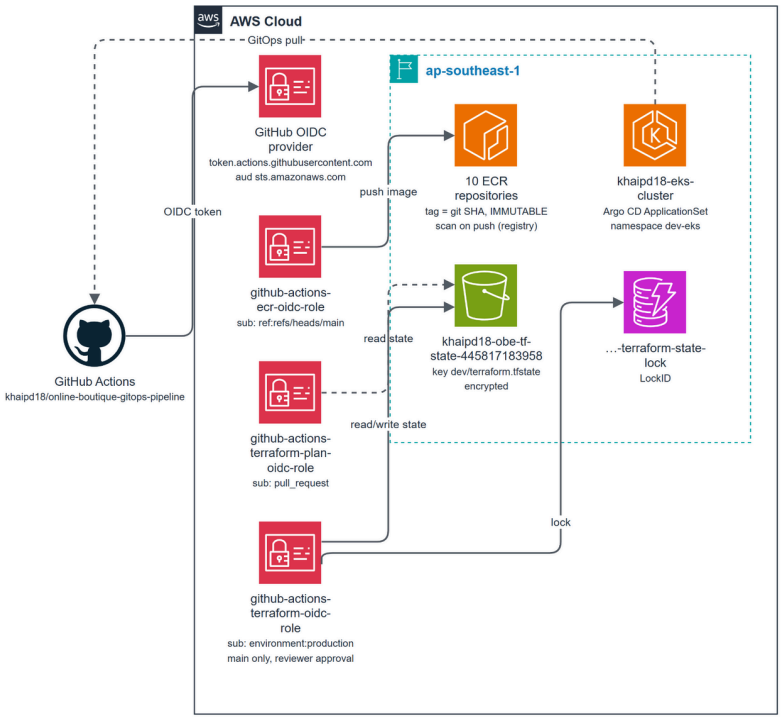
Cluster access	EKS access entries: the creator is admin, the console role is read-only (`AmazonEKSAdminViewPolicy`) [AWS]
ECR	10 repositories, immutable tags (tag = git SHA), scan on push, old images cleaned up by a lifecycle policy [AWS]
Terraform state	S3 bucket `khaipd18-obe-tf-state-445817183958` (versioned, encrypted) and DynamoDB lock table; created once by hand

Terraform is split into five modules: `vpc`, `eks`, `ecr`, `vpc-endpoints` and `github-oidc-role`. The account ID is never written in the code.

2.3 Access for CI (GitHub OIDC)

GitHub Actions gets short-lived AWS credentials from the GitHub OIDC provider. Each role trusts one exact `sub` value of the token [AWS]:

Role	Trusted sub	Used for
`github-actions-ecr-oidc-role`	`ref:refs/heads/main`	CI pushes images
`github-actions-terraform-plan-oidc-role`	`pull_request`	Read-only `terraform plan` on PRs
`github-actions-terraform-oidc-role`	`environment:production`	`terraform apply`, only after a reviewer approves



GitHub Actions roles			
Role	**Trusted sub (repo:<repo>:...)**	**Permissions**	**Used by**
github-actions-ecr-oidc-role	ref:refs/heads/main	ECR push/pull on the 10 repositories	5 service CI workflows
github-actions-terraform-plan-oidc-role	pull_request	ReadOnlyAccess + state read (plan -lock=false)	terraform.yaml on pull requests
github-actions-terraform-oidc-role	environment:production	AdministratorAccess + state read/write/lock	terraform.yaml apply job, push to main + approval

Figure 3: LLD: CI/CD and IAM OIDC

3 Kubernetes platform

All services run in namespace `dev-eks`. Every release uses the same Helm chart (`helm-charts/`); each service's values file only says what is different.

Service	Language	Port	Accepts calls from	CPU req/limit	Memory req/limit
frontend	Go	80 → 8080	any (LoadBalancer)	100m / 200m	64Mi / 256Mi
checkoutservice	Go	5050	frontend	100m / 200m	64Mi / 128Mi
productcatalogservice	Go	3550	frontend, checkout, recommendation	100m / 200m	64Mi / 128Mi
shippingservice	Go	50051	frontend, checkout	100m / 200m	64Mi / 128Mi
cartservice	C#	7070	frontend, checkout	200m / 300m	64Mi / 128Mi
adservice	Java	9555	frontend	200m / 300m	180Mi / 300Mi
currencyservice	Node.js	7000	frontend, checkout	100m / 200m	64Mi / 128Mi
paymentservice	Node.js	50051	checkout	100m / 200m	64Mi / 128Mi
emailservice	Python	8080	checkout	100m / 200m	64Mi / 128Mi
recommendationservice	Python	8080	frontend	100m / 200m	250Mi / 500Mi
redis-cart	Redis	6379	cartservice	100m / 200m	64Mi / 128Mi

Secure by default. Pods run as non-root with a read-only file system, no Linux capabilities and no service account token. Pod Security Admission (`restricted`) rejects any pod that does not follow these rules.

Who may call whom. Each service has a NetworkPolicy that only lets in the callers listed in its values file (`allowFrom`).

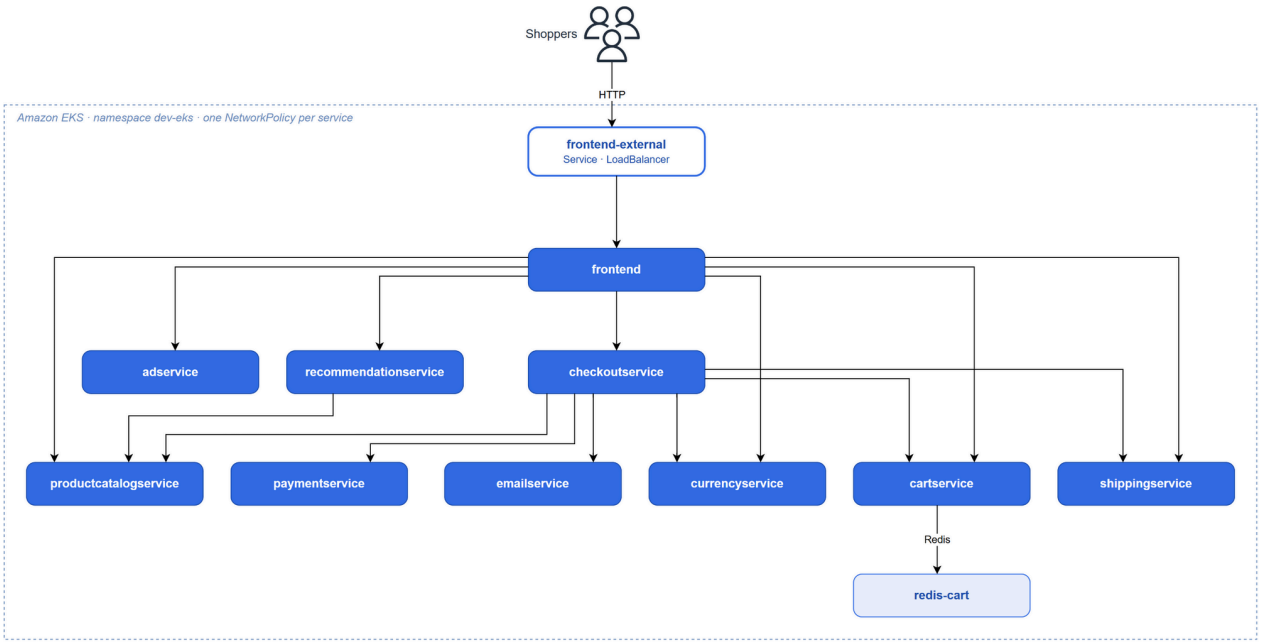


Figure 4: Calls allowed between services

Resilience. `frontend` runs 2–4 replicas with a CPU autoscaler (HPA), a PodDisruptionBudget and spreading across zones. The other services run one replica. `redis-cart` keeps carts in memory, so they are lost when it restarts.

Monitoring. Prometheus and Grafana (`kube-prometheus-stack`) in namespace `monitoring`; control plane logs in CloudWatch. No custom alerts yet.

4 Delivery pipeline

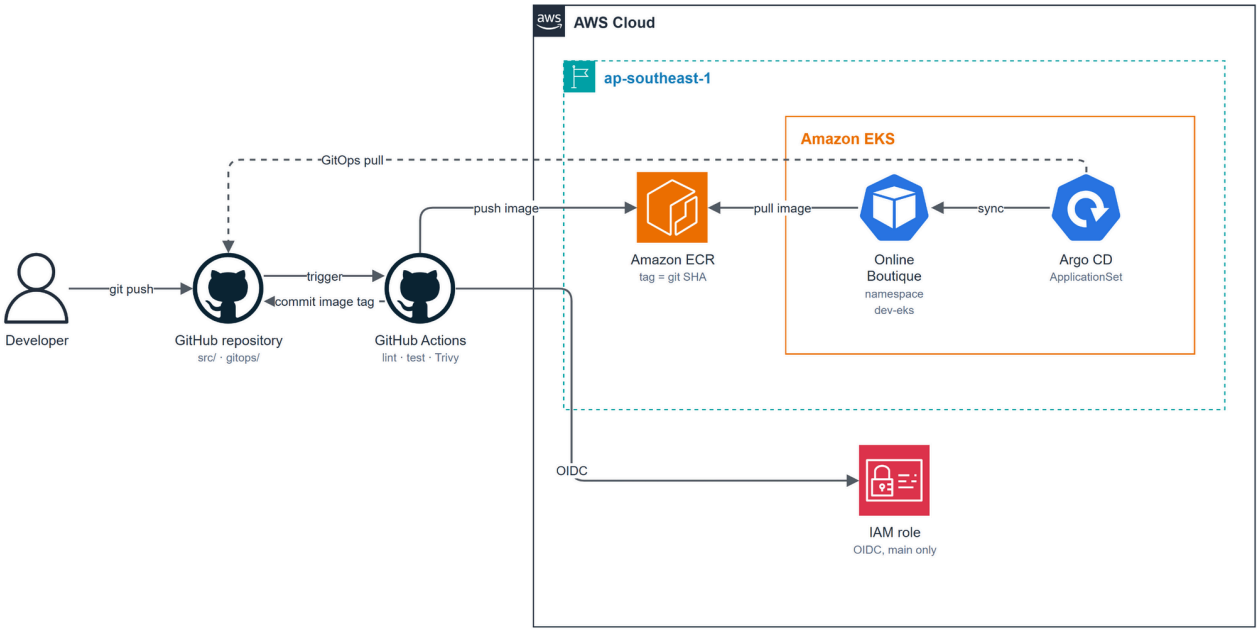


Figure 5: From commit to cluster

CI (one workflow per language). For each changed service: lint, test and dependency scan → build the image → Trivy scan (results in the GitHub Security tab, SBOM saved) → push to ECR with the git SHA as tag → write the tag into `gitops/dev-eks/` through a pull request that the bot opens and merges itself (`main` only accepts pull requests). Without the `AWS_ACCOUNT_ID` variable, CI still lints, tests, builds and scans, and skips the AWS steps.

Stack	Lint / format	Test	Dependency scan
Go (4 services)	golangci-lint v1.64.8 (blocking)	go test (blocking)	govulncheck v1.8.0 (warning)
.NET (cartservice)	dotnet format (warning)	dotnet test (blocking)	dotnet list package --vulnerable (blocking)
Java (adservice)	google-java-format (warning)	gradle test (blocking)	–
Node.js (2 services)	ESLint 8 (blocking)	–	npm audit (warning)
Python (2 services)	flake8 (blocking)	pytest (blocking when tests exist)	bandit (blocking)

Checks marked **warning** need changes in the upstream code, so they are shown but do not fail the run.

CD. One Argo CD ApplicationSet creates 12 Applications from `gitops/dev-eks/` and keeps them in sync: changes in Git are applied, changes made by hand are undone.

Infrastructure. A pull request that changes `terraform/` runs Checkov (blocking) and a read-only `terraform plan`. After the merge, `terraform apply` waits for a reviewer in the GitHub environment `production`. The `main` branch is protected by the ruleset `protect-main`.

5 Security

Risk	Control
Leaked access keys	No keys: GitHub OIDC for CI, IRSA for the VPC CNI
Unreviewed change reaches AWS	PRs get a read-only role; apply needs `main` and an approval
Misconfigured Terraform	Checkov blocks before plan and apply
Vulnerable images	Trivy before push; findings in code scanning; SBOM per image
Image overwritten	Immutable ECR tags
Compromised container	Non-root, read-only file system, no capabilities, PSA `restricted`
Moving between services	NetworkPolicy per service

Accepted findings. 13 Checkov findings are reviewed and kept in `terraform/.checkov.baseline`:

Check	Why it is accepted
`CKV_AWS_39` public EKS endpoint	Needed without a VPN; restrict later (roadmap)
`CKV_AWS_58`, `CKV_AWS_158` no customer KMS key	EKS and CloudWatch Logs already encrypt the data with AWS keys [AWS]
`CKV_AWS_229–232` open NACL ports	Filtering is done by security groups and NetworkPolicy
`CKV2_AWS_11` no VPC flow logs	Cost; enable for production
`CKV2_AWS_12` default security group	Not used by any resource

6 Design decisions

ID	Decision	Why / when to revisit
D1	GitHub OIDC instead of access keys	Nothing to leak or rotate.
D2	Apply needs approval in a GitHub environment	A merge alone cannot change infrastructure.
D3	One NAT Gateway for two AZs	Cheaper for dev. For production use a regional NAT gateway [AWS].
D4	One Helm chart for all services	Security defaults written once; a new service is one values file.
D5	ApplicationSet with a fixed list	What is deployed is visible in Git. Revisit with more environments.
D6	Image tag = git SHA, immutable	Every running image maps to a commit; rollback = previous SHA.
D7	Classic Load Balancer from a Service	No extra controller for dev. Revisit when TLS or WAF is needed.
D8	IRSA for the VPC CNI	Works everywhere. EKS Pod Identity is now simpler [AWS].
D9	DynamoDB state locking	Works with every Terraform version; move to S3 locking later.
D10	Upstream findings as warnings	The app code is not maintained here.
D11	Argo CD installed in the cluster	Free and fully controlled. EKS can also run Argo CD as a managed capability [AWS].
D12	Three nodes by default	The full stack needs about 37 pods; two `t3.medium` nodes are not enough [AWS].

7 Verification

Test	Result
Full deployment on EKS (2026-10-10)	74 resources on 3 nodes; Argo CD synced all 14 apps; a test order went through; PSA rejected a bad pod and NetworkPolicy blocked 2/2 disallowed connections; destroyed afterwards
kind cluster (local)	All pods ready; checkout works; HPA scaled 2 → 3 under load; PDB blocked draining the last `frontend` node
Terraform	`fmt`, `validate`, Checkov pass; plan = 74 resources
CI without AWS	All 5 language pipelines green; Trivy results for all 10 images
First EKS run (May 2026)	Images pushed over OIDC and synced by Argo CD (bot commits `14bc93f`, eb86a9a, 46e66d5)



Figure 6: Security controls checked on EKS: PSA rejects a non-compliant pod; NetworkPolicy blocks disallowed calls

8 Risks and roadmap

Known risks: one NAT Gateway is a single point of failure; ten services have one replica; carts are lost when `redis-cart` restarts; several images have known vulnerabilities; the EKS API endpoint is open to the internet (IAM still required).

Roadmap, in order:

1. Upgrade service dependencies, then make the vulnerability gates blocking.
2. Restrict the EKS public endpoint; narrow `ecr-endpoint-sg` to `tcp/443`.
3. Regional NAT gateway; AWS Load Balancer Controller with TLS.
4. External Secrets with AWS Secrets Manager; signed images pinned by digest.
5. Staging and production environments; SLO-based alerts.
6. S3 state locking; consider EKS Pod Identity and the Argo CD capability.

Appendix. Versions

Component	Version	Defined in
Terraform	1.14.8	terraform.yaml, README
AWS provider	6.39.0	terraform/versions.tf
Kubernetes (EKS)	1.35	terraform/variables.tf
VPC CNI add-on	v1.21.1-eksbuild.7	terraform/variables.tf
CoreDNS add-on	v1.13.2-eksbuild.4	terraform/variables.tf
kube-proxy add-on	v1.35.3-eksbuild.2	terraform/variables.tf
Metrics Server add-on	v0.9.0-eksbuild.11	terraform/variables.tf
kube-prometheus-stack	84.4.0	gitops/argocd/monitoring.yaml
Argo CD	stable manifest	README, step 3
Checkov	3.3.22	.github/workflows/*
Trivy	0.75.0	.github/actions/trivy-scan
Redis (cart store)	8.10.2	gitops/dev-eks/values-redis-cart.yaml